Applied Economics

Scraping Data

Pello Aspuru

Winter 2026/2027

Introduction

Applied economics **requires** data. → Data is often **scattered across multiple sources**.

- Last session: introduction to Dataset manipulation in R
  - Selecting data subsets.
  - Generating new variables.
  - Combining multiple datasets.
- This session: deeper dive into data collection and manipulation with R.
  - Types of data sources.
  - Obtain data from websites: Vibecoding a web scrapper.
  - Combining and saving the obtained data.

Introduction

Applied economics **requires** data. → Data is often **scattered across multiple sources**.

- Last session: introduction to Dataset manipulation in R
  - Selecting data subsets.
  - Generating new variables.
  - Combining multiple datasets.
- This session: deeper dive into data collection and manipulation with R.
  - Types of data sources.
  - Obtain data from websites: Vibecoding a web scrapper.
  - Combining and saving the obtained data.

Introduction

Applied economics **requires** data. → Data is often **scattered across multiple sources**.

- Last session: introduction to Dataset manipulation in R
  - Selecting data subsets.
  - Generating new variables.
  - Combining multiple datasets.
- This session: deeper dive into data collection and manipulation with R.
  - Types of data sources.
  - Obtain data from websites: Vibecoding a web scrapper.
  - Combining and saving the obtained data.

Three Sources of Data

Direct Download.

- Downloadable, free-access, ready-to-use files (CSV, Excel).
- Read into R with `read.csv()` or `read_excel()`.
- Easiest data source → Most exploited.

API (Application Programming Interface)

- A web service: User requests data, website returns it in a structured format.
- More restrictive: Requires user ID, download caps.

Unstructured Data

- Website information that is not organized as a dataset.
- e.g. the headlines of the main newspapers in Austria.

Three Sources of Data

Direct Download.

- Downloadable, free-access, ready-to-use files (CSV, Excel).
- Read into R with `read.csv()` or `read_excel()`.
- Easiest data source $\rightarrow$ Most exploited.

API (Application Programming Interface)

- A web service: User requests data, website returns it in a structured format.
- More restrictive: Requires user ID, download caps.

Unstructured Data

- Website information that is not organized as a dataset.
- e.g. the headlines of the main newspapers in Austria.

Three Sources of Data

Direct Download.

- Downloadable, free-access, ready-to-use files (CSV, Excel).
- Read into R with `read.csv()` or `read_excel()`.
- Easiest data source $\rightarrow$ Most exploited.

API (Application Programming Interface)

- A web service: User requests data, website returns it in a structured format.
- More restrictive: Requires user ID, download caps.

Unstructured Data

- Website information that is not organized as a dataset.
- e.g. the headlines of the main newspapers in Austria.

APIs - Example

The OpenWeather API provides current, forecast and historical weather data for any location:

```
https://api.openweathermap.org/data/2.5/weather?q=New%20York&units=metric&appid=YOUR_KEY
```

- Asks for the current weather in New York, in metric units: temperature, humidity, wind speed...
- `appid`: a personal key (free registration). Identifies user and limits the number of requests.

API Advantages

- Clean data, usually in JSON format.
- Well mantained, data still available when the website is redesigned.

APIs - Example

The OpenWeather API provides current, forecast and historical weather data for any location:

```
https://api.openweathermap.org/data/2.5/weather?q=New%20York&units=metric&appid=YOUR_KEY
```

- Asks for the current weather in New York, in metric units: temperature, humidity, wind speed...
- `appid`: a personal key (free registration). Identifies user and limits the number of requests.

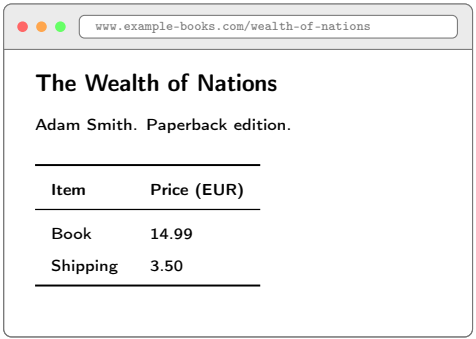
API Advantages

- Clean data, usually in JSON format.
- Well mantained, data still available when the website is redesigned.

Unstructured Data

Websites often **display** a lot of economically relevant information...

What you see (browser)



Underlying Code (HTML)

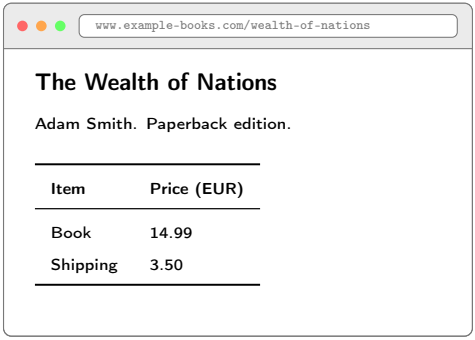
```
<html>
  <body>
    <h1>The Wealth of Nations</h1>
    <p>Adam Smith. Paperback edition.</p>
    <table>
      <tr><th>Item</th><th>Price (EUR)</th></tr>
      <tr><td>Book</td><td>14.99</td></tr>
      <tr><td>Shipping</td><td>3.50</td></tr>
    </table>
  </body>
</html>
```

Every element sits inside a **tag**: <h1> title, <p> paragraph, <table> table, <tr> row, <td> cell.

Webscraping

A **Webscraper** is a code script that...

1. Visits web pages automatically.
2. Reads the underlying HTML code.
3. Extracts the relevant information.
4. Stores information in a dataframe.



Webscraping

A **Webscraper** is a code script that...

1. Visits web pages automatically.
2. Reads the underlying HTML code.
3. Extracts the relevant information.
4. Stores information in a dataframe.

```
<html>
  <body>
    <h1>The Wealth of Nations</h1>
    <p>Adam Smith. Paperback edition.</p>
    <table>
      <tr><th>Item</th><th>Price (EUR)</th></tr>
      <tr><td>Book</td><td>14.99</td></tr>
      <tr><td>Shipping</td><td>3.50</td></tr>
    </table>
  </body>
</html>
```

Webscraping

A **Webscraper** is a code script that...

1. Visits web pages automatically.
2. Reads the underlying HTML code.
3. Extracts the relevant information.
4. Stores information in a dataframe.

```
<html>
  <body>
    <h1>The Wealth of Nations</h1>
    <p>Adam Smith. Paperback edition.</p>
    <table>
      <tr><th>Item</th><th>Price (EUR)</th></tr>
      <tr><td>Book</td><td>14.99</td></tr>
      <tr><td>Shipping</td><td>3.50</td></tr>
    </table>
  </body>
</html>
```

Webscraping

A **Webscraper** is a code script that...

1. Visits web pages automatically.
2. Reads the underlying HTML code.
3. Extracts the relevant information.
4. Stores information in a dataframe.

```
<html>
  <body>
    <h1>The Wealth of Nations</h1>
    <p>Adam Smith. Paperback edition.</p>
    <table>
      <tr><th>Item</th><th>Price (EUR)</th></tr>
      <tr><td>Book</td><td>14.99</td></tr>
      <tr><td>Shipping</td><td>3.50</td></tr>
    </table>
  </body>
</html>
```

⇓

product	shop	price_eur	shipping_eur
The Wealth of Nations	example-books.com	14.99	3.50

Programming Languages for Web Scraping

Webscrapers can be written in multiple programming languages:

- **Python:** the most popular choice for scraping (`requests`, `BeautifulSoup`, `Selenium`)
- **R:** less popular. Good enough for this course.

Package	What it does	Example functions
`rvest`	reads the HTML of a page and extracts its parts	`read_html()`, `html_elements()`, `html_text2()`, `html_table()`
`stringr`	cleans the extracted text, e.g. "312 ft (95 m)" → 95	`str_extract()`, `str_remove_all()`, `str_detect()`
`httr`	sends requests to websites and their APIs	`GET()`, `status_code()`, `content()`

Programming Languages for Web Scraping

Webscrapers can be written in multiple programming languages:

- **Python:** the most popular choice for scraping (`requests`, `BeautifulSoup`, `Selenium`)
- **R:** less popular. Good enough for this course.

Package	What it does	Example functions
`rvest`	reads the HTML of a page and extracts its parts	`read_html()`, `html_elements()`, `html_text2()`, `html_table()`
`stringr`	cleans the extracted text, e.g. "312 ft (95 m)" → 95	`str_extract()`, `str_remove_all()`, `str_detect()`
`httr`	sends requests to websites and their APIs	`GET()`, `status_code()`, `content()`

Vibecoding Webscrapers

Before LLMs...

- Writing a scraper used to be a **very tedious process**.
- Study the HTML of the pages, find the tags that contain the information, handle special cases...

After LLMs...

- **Vibecoding**: we describe what the scraping code should do, and let the LLM write it.
- Code written by an LLM often runs without errors, but returns incomplete or wrong information.
- Still need to audit: read the code, check its output, and ask for corrections.

Scrape responsibly: **respect the website's terms of use**.

Vibecoding Webscrapers

Before LLMs...

- Writing a scraper used to be a **very tedious process**.
- Study the HTML of the pages, find the tags that contain the information, handle special cases...

After LLMs...

- **Vibecoding**: we describe what the scraping code should do, and let the LLM write it.
- Code written by an LLM often runs without errors, but returns incomplete or wrong information.
- Still need to audit: read the code, check its output, and ask for corrections.

Scrape responsibly: **respect the website's terms of use**.

Vibecoding Webscrapers

Before LLMs...

- Writing a scraper used to be a **very tedious process**.
- Study the HTML of the pages, find the tags that contain the information, handle special cases...

After LLMs...

- **Vibecoding**: we describe what the scraping code should do, and let the LLM write it.
- Code written by an LLM often runs without errors, but returns incomplete or wrong information.
- Still need to audit: read the code, check its output, and ask for corrections.

Scrape responsibly: **respect the website's terms of use**.

Base Dataset: `flights`

Package `nycflights13`, dataset `flights`: 336,776 flights from New York City airports in 2013.

- `year`, `month`, `day`: date of departure.
- `dep_time`, `arr_time`: actual times, HHMM.
- `dep_delay`, `arr_delay`: delays, in minutes.
- `carrier`, `flight`, `tailnum`: airline, flight, plane.
- `origin`, `dest`: three-letter airport codes.
- `air_time`, `distance`: minutes in the air, miles.

```r
library(nycflights13)
data("flights")
dim(flights)
head(flights)
names(flights)
```

```
> dim(flights)
[1] 336776     19
```

```r
# Origin and destination airports
table(flights$origin, flights$dest)
```

Complementary Dataset: `airports`

Dataset `airports`: Records airports and their characteristics:

- `faa`: FAA code (e.g. `JFK`).
- `name`: name of the airport.
- `lat`, `lon`: latitude and longitude.
- `alt`: altitude above sea level, in feet.
- `tz`, `dst`, `tzone`: time zone information.

```
data("airports")
dim(airports)   # 1458 rows, 8 columns
head(airports)
names(airports)
```

Missing information:

- `n_runways`: Number of runways.
- `runway_length_m`: Length of runways.

Complementary Dataset: `airports`

Dataset `airports`: Records airports and their characteristics:

- `faa`: FAA code (e.g. `JFK`).
- `name`: name of the airport.
- `lat`, `lon`: latitude and longitude.
- `alt`: altitude above sea level, in feet.
- `tz`, `dst`, `tzone`: time zone information.

```
data("airports")
dim(airports)   # 1458 rows, 8 columns
head(airports)
names(airports)
```

Missing information:

- `n_runways`: Number of runways.
- `runway_length_m`: Length of runways.

Wikipedia Infobox Example



Washington Dulles International Airport

DULLES INTERNATIONAL

The Main Terminal at dusk in August 2011

IATA: IAD · **ICAO:** KIAD · **FAA LID:** IAD · **WMO:** 72403

Summary	
Airport type	Public
Owner	United States federal government
Operator	Metropolitan Washington Airports Authority
Serves	Washington metropolitan area
Location	Dulles, Virginia, U.S.
Opened	November 17, 1962; 63 years ago
Hub for	Southern Airways Express[1] United Airlines
Time zone	EST (UTC−05:00)
• Summer (DST)	EDT (UTC−04:00)
Elevation AMSL	312 ft (95 m)
Coordinates	38°56′40″N 077°27′21″W
Website	flydulles.com



Interactive map of Washington Dulles International Airport

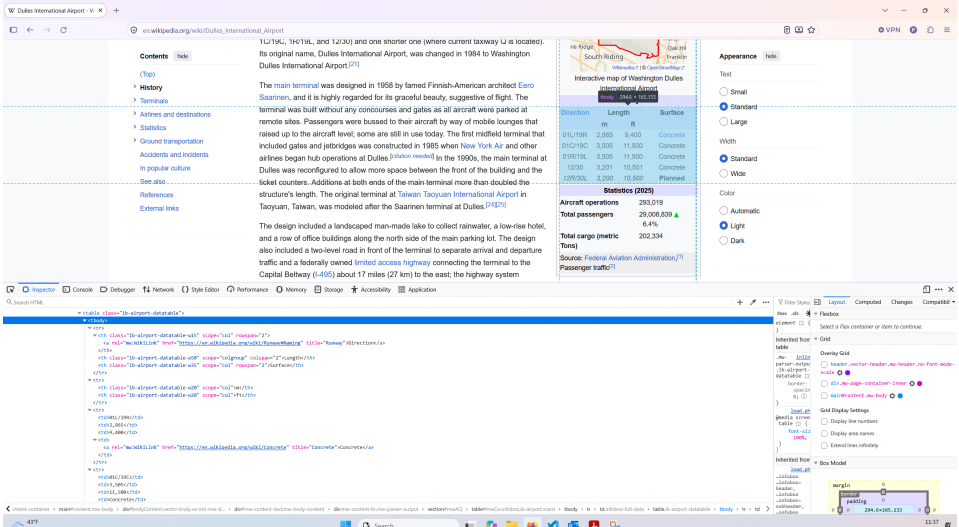
Runways			
Direction	Length		Surface
	m	ft	
01L/19R	2,865	9,400	Concrete
01C/19C	3,505	11,500	Concrete
01R/19L	3,505	11,500	Concrete
12/30	3,201	10,501	Concrete
12R/30L	*3,200*	*10,500*	**Planned**

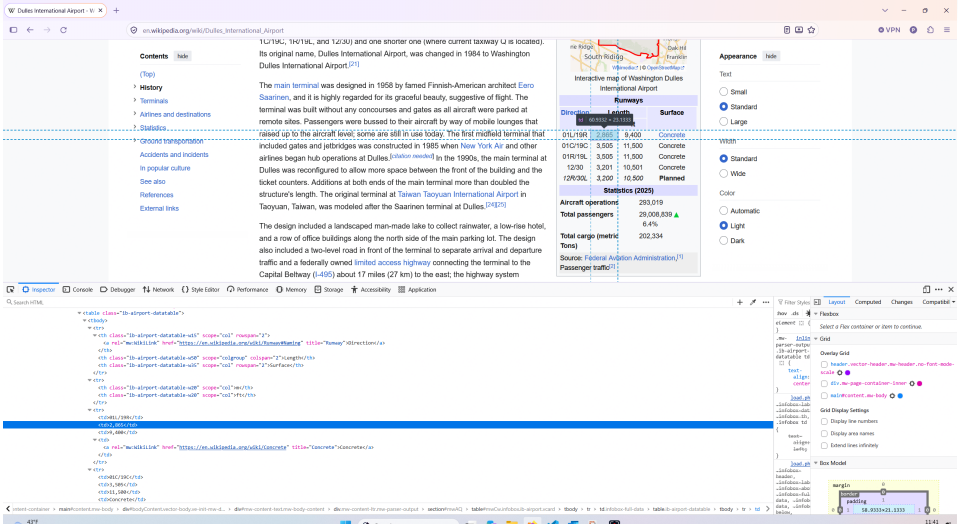
Statistics (2025)	
Aircraft operations	293,019
Total passengers	29,008,839 ▲ 6.4%
Total cargo (metric Tons)	202,334

Source: Federal Aviation Administration,[2] Passenger traffic[3]

Runways			
Direction	Length		Surface
	m	ft	
01L/19R	2,865	9,400	Concrete
01C/19C	3,505	11,500	Concrete
01R/19L	3,505	11,500	Concrete
12/30	3,201	10,501	Concrete
12R/30L	*3,200*	*10,500*	**Planned**

https://en.wikipedia.org/wiki/Dulles_International_Airport


Dulles International Airport - W
en.wikipedia.org/wiki/Dulles_International_Airport
Contents hide
(Top)
History
Terminals
Airlines and destinations
Statistics
Ground transportation
Accidents and incidents
In popular culture
See also
References
External links
Interactive map of Washington Dulles International Airport
Direction Length Surface
m ft
01L/19R 2,865 9,400 Concrete
01C/19C 3,505 11,500 Concrete
01R/19L 3,505 11,500 Concrete
12/30 3,201 10,501 Concrete
12R/30L 3,200 10,500 Planned
Statistics (2025)
Aircraft operations 293,019
Total passengers 29,008,839 ▲ 6.4%
Total cargo (metric Tons) 202,334
Source: Federal Aviation Administration,[1] Passenger traffic[2]
Appearance hide
Text
Small
Standard
Large
Width
Standard
Wide
Color
Automatic
Light
Dark
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
Layout
Computed
Changes
Compatibil
Flexbox
Select a Flex container or item to continue.
Grid
Overlay Grid
Grid Display Settings
Display line numbers
Display area names
Extend lines infinitely
Box Model
margin
border
padding
294.6×165.133
Search
Cloudy
11:37
11/11/2026


Dulles International Airport - W
en.wikipedia.org/wiki/Dulles_International_Airport
Contents hide
(Top)
History
Terminals
Airlines and destinations
Statistics
Ground transportation
Accidents and incidents
In popular culture
See also
References
External links
1C/19C, 1R/19L, and 12/30) and one shorter one (where current taxiway G is located). Its original name, Dulles International Airport, was changed in 1984 to Washington Dulles International Airport.[21]
The main terminal was designed in 1958 by famed Finnish-American architect Eero Saarinen, and it is highly regarded for its graceful beauty, suggestive of flight. The terminal was built without any concourses and gates as all aircraft were parked at remote sites. Passengers were bussed to their aircraft by way of mobile lounges that raised up to the aircraft level; some are still in use today. The first midfield terminal that included gates and jetbridges was constructed in 1985 when New York Air and other airlines began hub operations at Dulles.[citation needed] In the 1990s, the main terminal at Dulles was reconfigured to allow more space between the front of the building and the ticket counters. Additions at both ends of the main terminal more than doubled the structure's length. The original terminal at Taiwan Taoyuan International Airport in Taoyuan, Taiwan, was modeled after the Saarinen terminal at Dulles.[24][25]
The design included a landscaped man-made lake to collect rainwater, a low-rise hotel, and a row of office buildings along the north side of the main parking lot. The design also included a two-level road in front of the terminal to separate arrival and departure traffic and a federally owned limited access highway connecting the terminal to the Capital Beltway (I-495) about 17 miles (27 km) to the east; the highway system
Interactive map of Washington Dulles International Airport
Runways
Direction | Length | Surface
01L/19R | 2,869 m | 9,400 | Concrete
01C/19C | 3,505 | 11,500 | Concrete
01R/19L | 3,505 | 11,500 | Concrete
12/30 | 3,201 | 10,501 | Concrete
r2/R30L | 3,200 | 10,500 | Planned
Statistics (2025)
Aircraft operations | 293,019
Total passengers | 29,008,839 ▲ 6.4%
Total cargo (metric Tons) | 202,334
Source: Federal Aviation Administration,[1] Passenger traffic[2]
Appearance hide
Text
Small
Standard
Large
Width
Standard
Wide
Color
Automatic
Light
Dark
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application
Search HTML
Layout Computed Changes Compatibil
Flexbox
Select a Flex container or item to continue.
Grid
Overlay Grid
Grid Display Settings
Display line numbers
Display area names
Extend lines infinitely
Box Model
margin
border
padding
58.9333×21.3333
47°F Cloudy
Search
11:41
11/11/2026

Setting Up for Vibecoding

We only need the airports that appear in `flights`, as origin or destination:

```
# Keep only the airports that appear in flights, as origin or destination
airports <- airports[airports$faa %in% c(flights$origin, flights$dest), ]
nrow(airports)
head(airports$faa)
```

```
> nrow(airports)
[1] 103
> head(airports$faa)
[1] "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS"
```

The LLM Prompt

Ask your LLM of choice to write the scraper:

I am working on R. I have a list of unique airport codes from the flights dataset `airports$faa`, with elements of the list that look as follows: "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS"... I want to extract information about each airport from its Wikipedia page. Write an R script that loops through the list, scrapes the Wikipedia pages (e.g., `https://en.wikipedia.org/wiki/` + airport code), extracts: 1) the number of runways in the airport, 2) the length of each runway, in metres, from the Wikipedia infobox.

- A minimal version of the resulting scraper uses `rvest`.
- Airports whose page is not found are skipped.
- Runway lengths are stored as a single text, separated by semicolons.
- Outcome: `df_airports_wiki` dataset.

The LLM Prompt

Ask your LLM of choice to write the scraper:

> *I am working on R. I have a list of unique airport codes from the flights dataset `airports$faa`, with elements of the list that look as follows: "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS"... I want to extract information about each airport from its Wikipedia page. Write an R script that loops through the list, scrapes the Wikipedia pages (e.g., `https://en.wikipedia.org/wiki/` + airport code), extracts: 1) the number of runways in the airport, 2) the length of each runway, in metres, from the Wikipedia infobox.*

- A minimal version of the resulting scraper uses `rvest`.
- Airports whose page is not found are skipped.
- Runway lengths are stored as a single text, separated by semicolons.
- Outcome: `df_airports_wiki` dataset.

The First Vibecoded Scraper

```r
library(rvest)

# Empty dataframe to store the results, one row per airport
df_airports_wiki <- data.frame()

for (code in airports$faa) {

  # Read the Wikipedia page of the airport code; skip the code if the page does not exist
  page <- tryCatch(read_html(paste0("https://en.wikipedia.org/wiki/", code)),
                   error = function(e) NULL)
  if (is.null(page)) next

  # Rows of the runway table, inside the infobox at the top right of the page
  rows <- html_text2(html_elements(page, "table.infobox table tr"))

  # Runway rows start with a runway direction, e.g. "4L/22R  3,353  11,000  Asphalt"
  runways <- strsplit(rows[grepl("^[0-9]+[LCR]?/[0-9]+", rows)], "\t")

  # Lengths in metres: 2nd column, or 3rd if the table lists feet first
  m_col     <- if ("ft\tm" %in% rows) 3 else 2
  lengths_m <- sapply(runways, function(r) r[m_col])

  df_airports_wiki <- rbind(df_airports_wiki, data.frame(
    airport_code    = code,
    n_runways       = length(runways),
    runway_length_m = paste(lengths_m, collapse = "; ")
  ))

  Sys.sleep(0.5) # wait half a second between pages, to be polite to Wikipedia
}
```

Checking df_airports_wiki

Coverage of the n_runways variable seems to be perfect...

```
> table(is.na(df_airports_wiki$n_runways))
FALSE
  103
```

...but is it?

```
> table(df_airports_wiki$n_runways)
 0  1  2  3  4  5  6  8
85  2  5  2  6  1  1  1
```

85 of the 103 airports have 0 runways. Only a few airports are properly recorded:

```
> head(df_airports_wiki[df_airports_wiki$n_runways != 0, ])
   airport_code n_runways                                runway_length_m
8           BDL         2                                   2,899; 2,087
19          CAK         2                                   2,317; 2,501
32          DTW         6 3,659; 3,048; 3,048; 2,591; 2,654; 2,591
33          EGE         1                                          2,743
```

Checking df_airports_wiki

Coverage of the n_runways variable seems to be perfect...

```
> table(is.na(df_airports_wiki$n_runways))
FALSE
  103
```

...but is it?

```
> table(df_airports_wiki$n_runways)
 0  1  2  3  4  5  6  8
85  2  5  2  6  1  1  1
```

85 of the 103 airports have 0 runways. Only a few airports are properly recorded:

```
> head(df_airports_wiki[df_airports_wiki$n_runways != 0, ])
   airport_code n_runways                                runway_length_m
8           BDL          2                                   2,899; 2,087
19          CAK          2                                   2,317; 2,501
32          DTW          6 3,659; 3,048; 3,048; 2,591; 2,654; 2,591
33          EGE          1                                          2,743
```

Checking df_airports_wiki

Coverage of the n_runways variable seems to be perfect...

```
> table(is.na(df_airports_wiki$n_runways))
FALSE
  103
```

...but is it?

```
> table(df_airports_wiki$n_runways)
 0  1  2  3  4  5  6  8
85  2  5  2  6  1  1  1
```

85 of the 103 airports have 0 runways. Only a few airports are properly recorded:

```
> head(df_airports_wiki[df_airports_wiki$n_runways != 0, ])
   airport_code n_runways                          runway_length_m
8           BDL          2                             2,899; 2,087
19          CAK          2                             2,317; 2,501
32          DTW          6 3,659; 3,048; 3,048; 2,591; 2,654; 2,591
33          EGE          1                                    2,743
```

Sanity Check: Dulles International Airport

Check the output against a case study.

- Washington Dulles International Airport (IAD)
- **4 runways**: 3,505; 3,505; 3,200; 3,200 metres.

Did the scraper get it right?

Runways			
Direction	Length		Surface
	m	ft	
01L/19R	2,865	9,400	Concrete
01C/19C	3,505	11,500	Concrete
01R/19L	3,505	11,500	Concrete
12/30	3,201	10,501	Concrete
12R/30L	*3,200*	*10,500*	**Planned**

```
> print(df_airports_wiki[df_airports_wiki$airport_code == "IAD", ])
   airport_code n_runways runway_length_m
43          IAD         0
```

According to the scraper, Dulles has no runways.

Sanity Check: Dulles International Airport

Check the output against a case study.

- Washington Dulles International Airport (IAD)
- **4 runways**: 3,505; 3,505; 3,200; 3,200 metres.

Did the scraper get it right?

Runways			
Direction	Length		Surface
	m	ft	
01L/19R	2,865	9,400	Concrete
01C/19C	3,505	11,500	Concrete
01R/19L	3,505	11,500	Concrete
12/30	3,201	10,501	Concrete
12R/30L	*3,200*	*10,500*	**Planned**

```
> print(df_airports_wiki[df_airports_wiki$airport_code == "IAD", ])
   airport_code n_runways runway_length_m
43          IAD         0
```

According to the scraper, Dulles has no runways.

Sanity Check: Dulles International Airport

Check the output against a case study.

- Washington Dulles International Airport (IAD)
- **4 runways**: 3,505; 3,505; 3,200; 3,200 metres.

Did the scraper get it right?

Runways			
Direction	Length		Surface
	m	ft	
01L/19R	2,865	9,400	Concrete
01C/19C	3,505	11,500	Concrete
01R/19L	3,505	11,500	Concrete
12/30	3,201	10,501	Concrete
12R/30L	*3,200*	*10,500*	**Planned**

```
> print(df_airports_wiki[df_airports_wiki$airport_code == "IAD", ])
   airport_code n_runways runway_length_m
43          IAD         0
```

According to the scraper, Dulles has no runways.

Lesson from the First Attempt

- The vibecoded scraper recorded the runway information for a **few airports**...
- ...but **most airports** were not recorded properly.

Danger: the failure is easy to miss

- Even when the scraper found no information, it filled in `n_runways` with 0.
- Every airport has a value: at first glance, the scraper seems to have done a great job.

A script that runs without errors and returns a complete dataset is not necessarily correct.

Lesson from the First Attempt

- The vibecoded scraper recorded the runway information for a **few airports**...
- ...but **most airports** were not recorded properly.

Danger: the failure is easy to miss

- Even when the scraper found no information, it filled in `n_runways` with 0.
- Every airport has a value: at first glance, the scraper seems to have done a great job.

A script that runs without errors and returns a complete dataset is not necessarily correct.

Lesson from the First Attempt

- The vibecoded scraper recorded the runway information for a **few airports**...
- ...but **most airports** were not recorded properly.

Danger: the failure is easy to miss

- Even when the scraper found no information, it filled in `n_runways` with 0.
- Every airport has a value: at first glance, the scraper seems to have done a great job.

A script that runs without errors and returns a complete dataset is not necessarily correct.

Refining the Prompt

List the airports that were not found in the first attempt:

```
> df_airports_not_found <- df_airports_wiki$airport_code[df_airports_wiki$n_runways == 0]
> print(df_airports_not_found)
 [1] "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS" "AVL" "BGR" "BHM" "BNA" "BOS" "BTV"
[13] "BUF" "BUR" "BWI" "BZN" "CAE" "CHO" "CHS" "CLE" "CLT" "CMH" "CRW" "CVG"
...
[85] "XNA"
```

Generate a new prompt, asking the LLM to improve the scraper:

The previous scraper missed the following airports: [paste the list of missed airports here]. Moreover, I want that, if you are unable to find a specific variable, such as the number of runways `n_runways`, you record "NA" instead of 0. If the number of runways is NA, record the runway length variable `runway_length_m` as "NA". Generate a new version of the scraper.

Refining the Prompt

List the airports that were not found in the first attempt:

```
> df_airports_not_found <- df_airports_wiki$airport_code[df_airports_wiki$n_runways == 0]
> print(df_airports_not_found)
 [1] "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS" "AVL" "BGR" "BHM" "BNA" "BOS" "BTV"
[13] "BUF" "BUR" "BWI" "BZN" "CAE" "CHO" "CHS" "CLE" "CLT" "CMH" "CRW" "CVG"
...
[85] "XNA"
```

Generate a new prompt, asking the LLM to improve the scraper:

The previous scraper missed the following airports: [paste the list of missed airports here]. Moreover, I want that, if you are unable to find a specific variable, such as the number of runways `n_runways`, you record "NA" instead of 0. If the number of runways is NA, record the runway length variable `runway_length_m` as "NA". Generate a new version of the scraper.

What Changed in the Scraper

New 1: instead of guessing the page address, ask Wikipedia's search engine for "*code* airport" and take the title of the first result:

```
search_url <- paste0("https://en.wikipedia.org/w/api.php?action=query&list=search",
                     "&format=json&srsearch=", code, "%20airport")
title <- tryCatch(content(GET(search_url))$query$search[[1]]$title,
                  error = function(e) NULL)
page <- if (is.null(title)) NULL else
  tryCatch(read_html(paste0("https://en.wikipedia.org/wiki/", URLencode(title))),
           error = function(e) NULL)
```

New 2: if the page is not found, keep the airport in the dataset with `NA` values:

```
if (is.null(page)) {
  df_airports_wiki2 <- rbind(df_airports_wiki2, data.frame(
    airport_code = code, airport_name = NA, n_runways = NA, runway_length_m = NA))
  next
}
```

New 3: `NA` instead of 0 if no runways are found, and then also `NA` for their lengths:

```
n_runways       <- if (length(runways) == 0) NA else length(runways)
runway_length_m <- if (is.na(n_runways)) NA else paste(lengths_m, collapse = "; ")
```

The rest of the scraper (runway table, lengths in metres) is unchanged.

What Changed in the Scraper

New 1: instead of guessing the page address, ask Wikipedia's search engine for "*code* airport" and take the title of the first result:

```
search_url <- paste0("https://en.wikipedia.org/w/api.php?action=query&list=search",
                     "&format=json&srsearch=", code, "%20airport")
title <- tryCatch(content(GET(search_url))$query$search[[1]]$title,
                  error = function(e) NULL)
page <- if (is.null(title)) NULL else
  tryCatch(read_html(paste0("https://en.wikipedia.org/wiki/", URLencode(title))),
           error = function(e) NULL)
```

New 2: if the page is not found, keep the airport in the dataset with `NA` values:

```
if (is.null(page)) {
  df_airports_wiki2 <- rbind(df_airports_wiki2, data.frame(
    airport_code = code, airport_name = NA, n_runways = NA, runway_length_m = NA))
  next
}
```

New 3: `NA` instead of 0 if no runways are found, and then also `NA` for their lengths:

```
n_runways       <- if (length(runways) == 0) NA else length(runways)
runway_length_m <- if (is.na(n_runways)) NA else paste(lengths_m, collapse = "; ")
```

The rest of the scraper (runway table, lengths in metres) is unchanged.

What Changed in the Scraper

New 1: instead of guessing the page address, ask Wikipedia's search engine for "*code* airport" and take the title of the first result:

```
search_url <- paste0("https://en.wikipedia.org/w/api.php?action=query&list=search",
                     "&format=json&srsearch=", code, "%20airport")
title <- tryCatch(content(GET(search_url))$query$search[[1]]$title,
                  error = function(e) NULL)
page <- if (is.null(title)) NULL else
  tryCatch(read_html(paste0("https://en.wikipedia.org/wiki/", URLencode(title))),
           error = function(e) NULL)
```

New 2: if the page is not found, keep the airport in the dataset with `NA` values:

```
if (is.null(page)) {
  df_airports_wiki2 <- rbind(df_airports_wiki2, data.frame(
    airport_code = code, airport_name = NA, n_runways = NA, runway_length_m = NA))
  next
}
```

New 3: `NA` instead of 0 if no runways are found, and then also `NA` for their lengths:

```
n_runways       <- if (length(runways) == 0) NA else length(runways)
runway_length_m <- if (is.na(n_runways)) NA else paste(lengths_m, collapse = "; ")
```

The rest of the scraper (runway table, lengths in metres) is unchanged.

Results of the Second Attempt

```
> head(df_airports_wiki2)
  airport_code n_runways      runway_length_m
1          ABQ          3 3,048; 4,204; 1,829
2          ACK          2        1,921; 1,372
3          ALB          2        2,591; 2,195
4          ANC          3 3,780; 3,312; 3,231
```

```
> table(is.na(df_airports_wiki2$n_runways))
FALSE
  103
> table(df_airports_wiki2$n_runways)
 1  2  3  4  5  6  7  8
12 36 27 19  4  3  1  1
```

No more airports with 0 runways. Sanity check: what about Dulles?

```
> print(df_airports_wiki2[df_airports_wiki2$airport_code == "IAD",
+                         c("airport_code", "n_runways", "runway_length_m")])
   airport_code n_runways                    runway_length_m
43          IAD          5 2,865; 3,505; 3,505; 3,201; 3,200
```

Much better, but is it right? Compare with the Wikipedia infobox.

Results of the Second Attempt

```
> head(df_airports_wiki2)
  airport_code n_runways      runway_length_m
1          ABQ          3 3,048; 4,204; 1,829
2          ACK          2         1,921; 1,372
3          ALB          2         2,591; 2,195
4          ANC          3 3,780; 3,312; 3,231
```

```
> table(is.na(df_airports_wiki2$n_runways))
FALSE
  103
> table(df_airports_wiki2$n_runways)
 1  2  3  4  5  6  7  8
12 36 27 19  4  3  1  1
```

No more airports with 0 runways. Sanity check: what about Dulles?

```
> print(df_airports_wiki2[df_airports_wiki2$airport_code == "IAD",
+                         c("airport_code", "n_runways", "runway_length_m")])
   airport_code n_runways                  runway_length_m
43          IAD          5 2,865; 3,505; 3,505; 3,201; 3,200
```

Much better, but is it right? Compare with the Wikipedia infobox.

Results of the Second Attempt

```
> head(df_airports_wiki2)
  airport_code n_runways      runway_length_m
1          ABQ          3 3,048; 4,204; 1,829
2          ACK          2         1,921; 1,372
3          ALB          2         2,591; 2,195
4          ANC          3 3,780; 3,312; 3,231
```

```
> table(is.na(df_airports_wiki2$n_runways))
FALSE
  103
> table(df_airports_wiki2$n_runways)
 1  2  3  4  5  6  7  8
12 36 27 19  4  3  1  1
```

No more airports with 0 runways. Sanity check: what about Dulles?

```
> print(df_airports_wiki2[df_airports_wiki2$airport_code == "IAD",
+                         c("airport_code", "n_runways", "runway_length_m")])
   airport_code n_runways                  runway_length_m
43          IAD          5 2,865; 3,505; 3,505; 3,201; 3,200
```

Much better, but is it right? Compare with the Wikipedia infobox.

Merging Twice: Airport of Origin

- `df_airports_wiki2` is a complement: we merge it with the base dataset `flights`.
- Each flight involves two airports: merge **twice**, once for the origin and once for the destination.
- Rename the variables with the prefix `origin_`, and the key as `origin`, as in `flights`:

```r
library(dplyr)

# Airport information, referring to the airport of origin
df_origin <- df_airports_wiki2 |>
  select(origin                  = airport_code,
         origin_n_runways        = n_runways,
         origin_runway_length_m = runway_length_m)

df_flights <- flights |>
  left_join(df_origin, by = "origin")
```

A left join keeps every flight (336,776 rows). Only three origin airports:

```
> table(df_flights$origin,
+       df_flights$origin_n_runways)
          2      3      4
  EWR     0 120835      0
  JFK     0      0 111279
  LGA 104662      0      0
```

Merging Twice: Airport of Origin

- `df_airports_wiki2` is a complement: we merge it with the base dataset `flights`.
- Each flight involves two airports: merge **twice**, once for the origin and once for the destination.
- Rename the variables with the prefix `origin_`, and the key as `origin`, as in `flights`:

```
library(dplyr)

# Airport information, referring to the airport of origin
df_origin <- df_airports_wiki2 |>
  select(origin                 = airport_code,
         origin_n_runways       = n_runways,
         origin_runway_length_m = runway_length_m)

df_flights <- flights |>
  left_join(df_origin, by = "origin")
```

A left join keeps every flight (336,776 rows). Only three origin airports:

```
> table(df_flights$origin,
+       df_flights$origin_n_runways)
            2      3      4
  EWR       0 120835      0
  JFK       0      0 111279
  LGA  104662      0      0
```

What the Left Join Does

`flights`: **336,776 rows, 19 columns**

carrier	flight	origin	dest	...
UA	1545	EWR	IAH	...
UA	1714	LGA	IAH	...
AA	1141	JFK	MIA	...
B6	725	JFK	BQN	...
⋮				

`df_origin`: **103 rows, 3 columns**

origin	origin_n_runways	origin_runway_length_m
EWR	3	3,353; 3,048; 2,050
JFK	4	3,682; 2,560; 3,048; 4,423
LGA	2	2,134; 2,134
⋮		

```
left_join(df_origin, by = "origin")
```

carrier	flight	origin	dest	...	origin_n_runways	origin_runway_length_m
UA	1545	EWR	IAH	...	3	3,353; 3,048; 2,050
UA	1714	LGA	IAH	...	2	2,134; 2,134
AA	1141	JFK	MIA	...	4	3,682; 2,560; 3,048; 4,423
B6	725	JFK	BQN	...	4	3,682; 2,560; 3,048; 4,423

`df_flights`: **336,776 rows, 21 columns**

What the Left Join Does

`flights`: **336,776 rows, 19 columns**

carrier	flight	origin	dest	...
UA	1545	EWR	IAH	...
UA	1714	LGA	IAH	...
AA	1141	JFK	MIA	...
B6	725	JFK	BQN	...
.				
.				
.				

`df_origin`: **103 rows, 3 columns**

origin	origin_n_runways	origin_runway_length_m
EWR	3	3,353; 3,048; 2,050
JFK	4	3,682; 2,560; 3,048; 4,423
LGA	2	2,134; 2,134
.		
.		

```
left_join(df_origin, by = "origin")
```

carrier	flight	origin	dest	...	origin_n_runways	origin_runway_length_m
UA	1545	EWR	IAH	...	3	3,353; 3,048; 2,050
UA	1714	LGA	IAH	...	2	2,134; 2,134
AA	1141	JFK	MIA	...	4	3,682; 2,560; 3,048; 4,423
B6	725	JFK	BQN	...	4	3,682; 2,560; 3,048; 4,423

`df_flights`: **336,776 rows, 21 columns**

Merging Twice: Airport of Destination

Same steps, with the prefix `dest_`:

```
# Airport information, referring to the airport of destination
df_dest <- df_airports_wiki2 |>
  select(dest                = airport_code,
         dest_n_runways      = n_runways,
         dest_runway_length_m = runway_length_m)

df_flights <- df_flights |>
  left_join(df_dest, by = "dest")

dim(df_flights)   # 336776 rows, 23 columns
```

Coverage of the runway variables (rows: destination missing; columns: origin missing):

```
> table(is.na(df_flights$dest_n_runways), is.na(df_flights$origin_n_runways))
       FALSE
  FALSE 329174
  TRUE    7602
```

7,602 flights have no information on the airport of destination.

Merging Twice: Airport of Destination

Same steps, with the prefix `dest_`:

```
# Airport information, referring to the airport of destination
df_dest <- df_airports_wiki2 |>
  select(dest                = airport_code,
         dest_n_runways      = n_runways,
         dest_runway_length_m = runway_length_m)

df_flights <- df_flights |>
  left_join(df_dest, by = "dest")

dim(df_flights)   # 336776 rows, 23 columns
```

Coverage of the runway variables (rows: destination missing; columns: origin missing):

```
> table(is.na(df_flights$dest_n_runways), is.na(df_flights$origin_n_runways))
         FALSE
  FALSE 329174
  TRUE    7602
```

7,602 flights have no information on the airport of destination.

What the Second Left Join Does

`df_flights`: **336,776 rows, 21 columns**

flight	origin	dest	...	origin_n_runways
1545	EWR	IAH	...	3
1714	LGA	IAH	...	2
1141	JFK	MIA	...	4
725	JFK	BQN	...	4
⋮				

`df_dest`: **103 rows, 3 columns (no BQN)**

dest	dest_n_runways	dest_runway_length_m
IAH	5	3,658; 3,048; 3,048; 2,743; 2,866
MIA	4	2,621; 3,202; 3,967; 2,853
⋮		

```
left_join(df_dest, by = "dest")
```

flight	origin	dest	...	origin_n_runways	dest_n_runways	dest_runway_length_m
1545	EWR	IAH	...	3	5	3,658; 3,048; 3,048; 2,743; 2,866
1714	LGA	IAH	...	2	5	3,658; 3,048; 3,048; 2,743; 2,866
1141	JFK	MIA	...	4	4	2,621; 3,202; 3,967; 2,853
725	JFK	BQN	...	4	NA	NA

`df_flights`: 336,776 rows, 23 columns. BQN has no match: kept, with `NA`.

What the Second Left Join Does

`df_flights`: **336,776 rows, 21 columns**

flight	origin	dest	...	origin_n_runways
1545	EWR	IAH	...	3
1714	LGA	IAH	...	2
1141	JFK	MIA	...	4
725	JFK	BQN	...	4
⋮				

`df_dest`: **103 rows, 3 columns (no BQN)**

dest	dest_n_runways	dest_runway_length_m
IAH	5	3,658; 3,048; 3,048; 2,743; 2,866
MIA	4	2,621; 3,202; 3,967; 2,853
⋮		

`left_join(df_dest, by = "dest")`

flight	origin	dest	...	origin_n_runways	dest_n_runways	dest_runway_length_m
1545	EWR	IAH	...	3	5	3,658; 3,048; 3,048; 2,743; 2,866
1714	LGA	IAH	...	2	5	3,658; 3,048; 3,048; 2,743; 2,866
1141	JFK	MIA	...	4	4	2,621; 3,202; 3,967; 2,853
725	JFK	BQN	...	4	NA	NA

`df_flights`: **336,776 rows, 23 columns. BQN has no match: kept, with `NA`.**

Saving the Dataset

Save the enriched dataset to a CSV file for later use. Unless told otherwise, R saves it in the working directory:

```
# Print the working directory
getwd()

# Save dataset df_flights
write.csv(df_flights, "data/df_flights_enriched.csv", row.names = FALSE)
```

Summary

Today

1. Data from the web: direct download, APIs, web scraping.
2. Vibecoding a scraper with an LLM, and **checking** its output: coverage tables, and a known case (Dulles).
3. Refining the prompt until the output is reliable.
4. Merging the scraped data with the base dataset, twice: origin and destination.

Never trust a dataset just because it looks complete.

Next class: once our data is assembled and cleaned, we start exploring it with summary tables and visualizations of the relationships between economically relevant variables.

Summary

Today

1. Data from the web: direct download, APIs, web scraping.
2. Vibecoding a scraper with an LLM, and **checking** its output: coverage tables, and a known case (Dulles).
3. Refining the prompt until the output is reliable.
4. Merging the scraped data with the base dataset, twice: origin and destination.

Never trust a dataset just because it looks complete.

Next class: once our data is assembled and cleaned, we start exploring it with summary tables and visualizations of the relationships between economically relevant variables.

Summary

Today

1. Data from the web: direct download, APIs, web scraping.
2. Vibecoding a scraper with an LLM, and **checking** its output: coverage tables, and a known case (Dulles).
3. Refining the prompt until the output is reliable.
4. Merging the scraped data with the base dataset, twice: origin and destination.

Never trust a dataset just because it looks complete.

Next class: once our data is assembled and cleaned, we start exploring it with summary tables and visualizations of the relationships between economically relevant variables.